

项目一：打字游戏

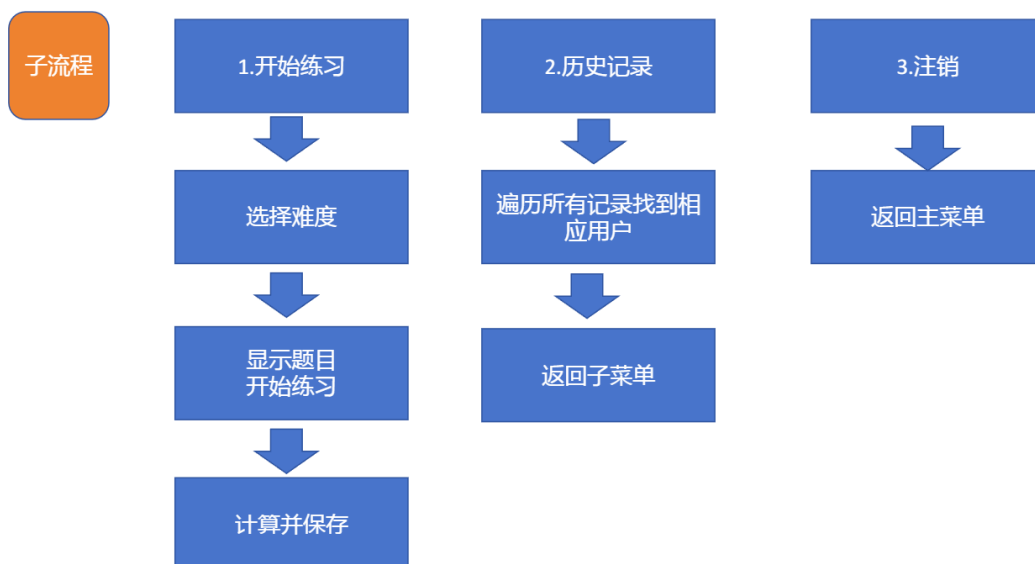
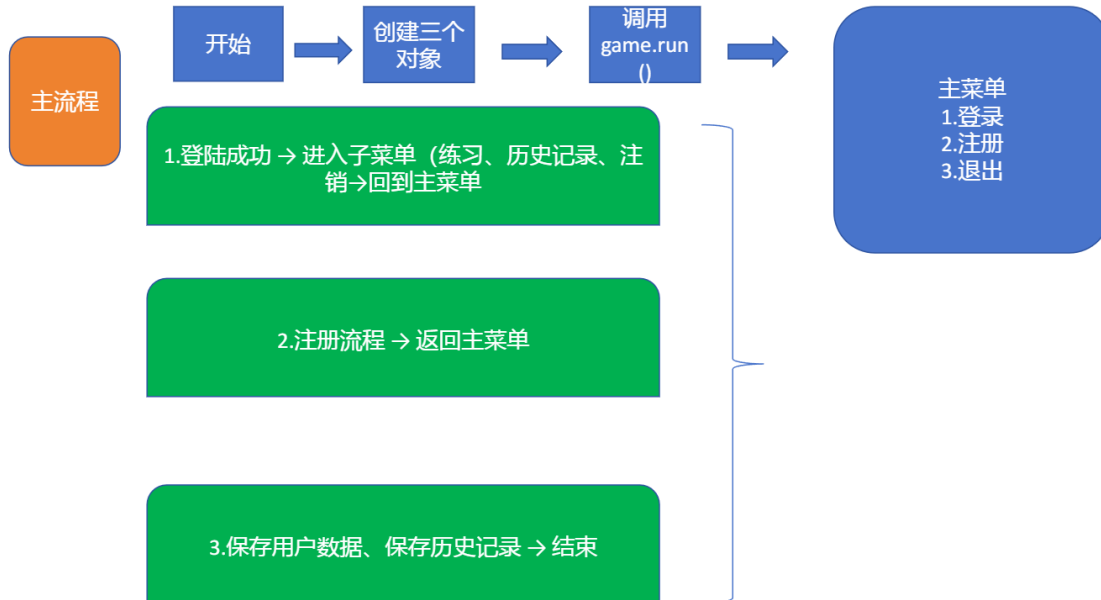
卢耀琪

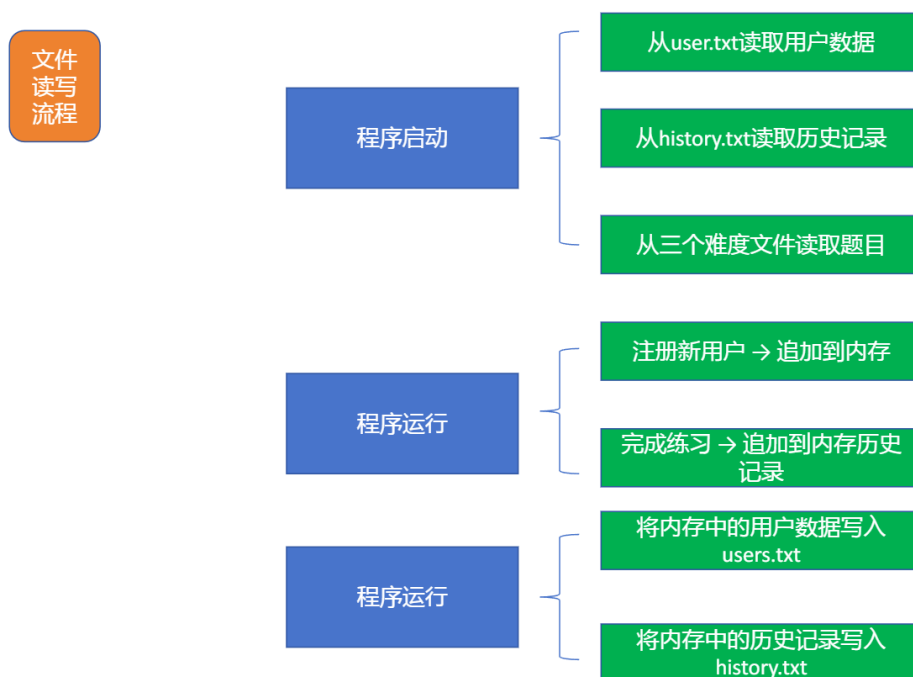
计算机金融实验班

2026 年 5 月 25 日

1 程序设计

1.1 程序设计流程





1.2 主要数据结构设计

1.2.1 用户数据结构

```

1 struct user
2 {
3     string username;
4     string password;
5 };
  
```

- 将用户名和密码这两个关联信息打包在一起，比用两个独立的数组更清晰，代码可读性高，也方便作为整体传递给函数。

```

1 struct History
2 {
3     string username;
4     time_t time;
5     int dif;
6     double ac;
7     double speed;
8 };
  
```

- 包含用户名、练习时间（时间戳）、难度、正确率、打字速度。用来记录每一次打字练习的成绩。同样用一个数组存储，最多 500 条。数组顺序存储，方便遍历和按用户查找。

```

1  vector<string> easyQuestions;
2  vector<string> mediumQuestions;
3  vector<string> hardQuestions;

```

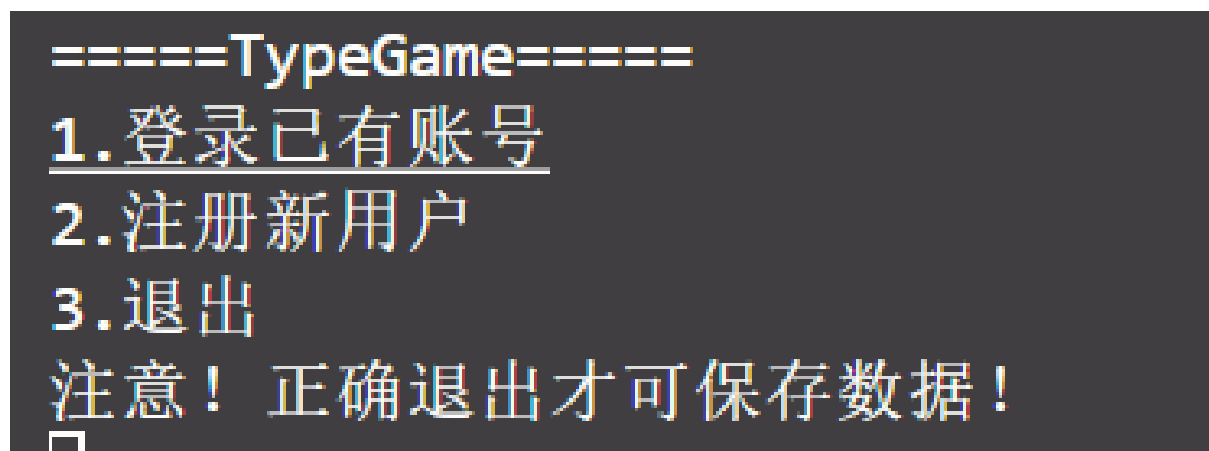
- 题目数量可能随时增减，可以自动扩容，不需要预先知道题目总数。相比固定数组，更灵活，不会因题目数量超过预设值而报错。

1.3 程序功能及对应的函数头

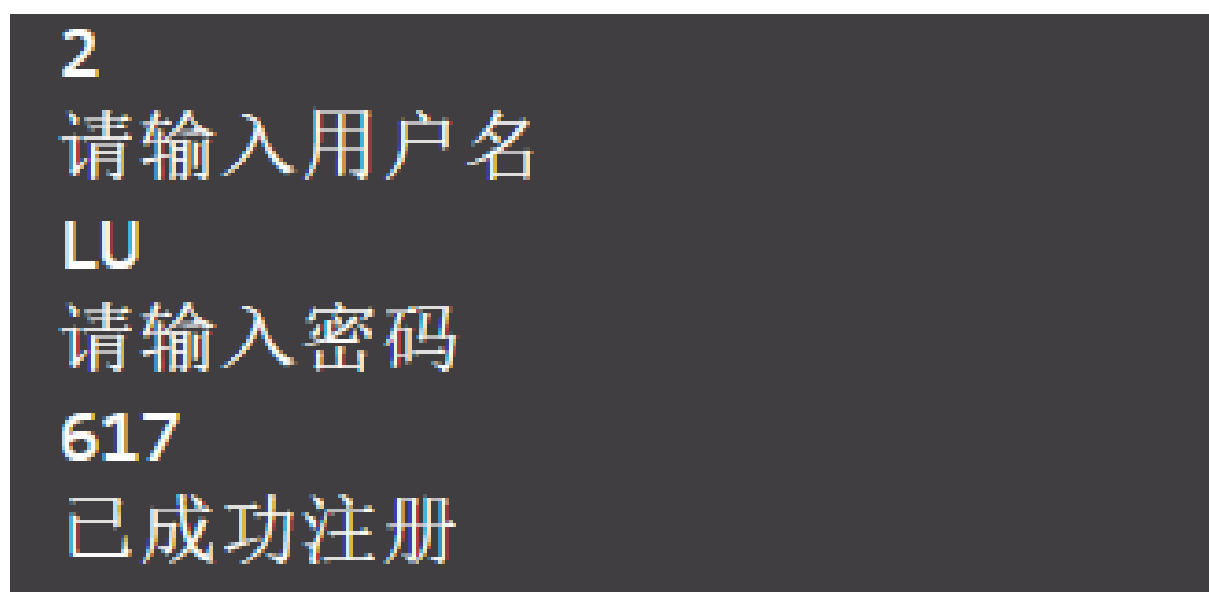
模块	功能	函数头
用户管理	加载用户数据	void loadUsers();
	保存用户数据	void saveUsers();
	注册用户	bool registerUser();
	用户登录	bool login(std::string& loggedUsername);
历史记录管理	加载历史记录	void loadHistories();
	保存历史记录	void saveHistories();
	添加记录	void addRecord(const std::string& username, time_t t, int dif, double ac, double speed);
	显示历史	void showHistory(const std::string& username);
游戏核心	主循环	void run();
	加载题目	void loadQuestions(const std::string& filename, std::vector<std::string>& container);
	随机取题	std::string getText(int dif);
	开始练习	void start(const std::string& username);
	子菜单	void showMenu(const std::string& loggedUser);
	清理输入	void cleanInput();
程序入口	主函数	int main();

2 程序运行截图

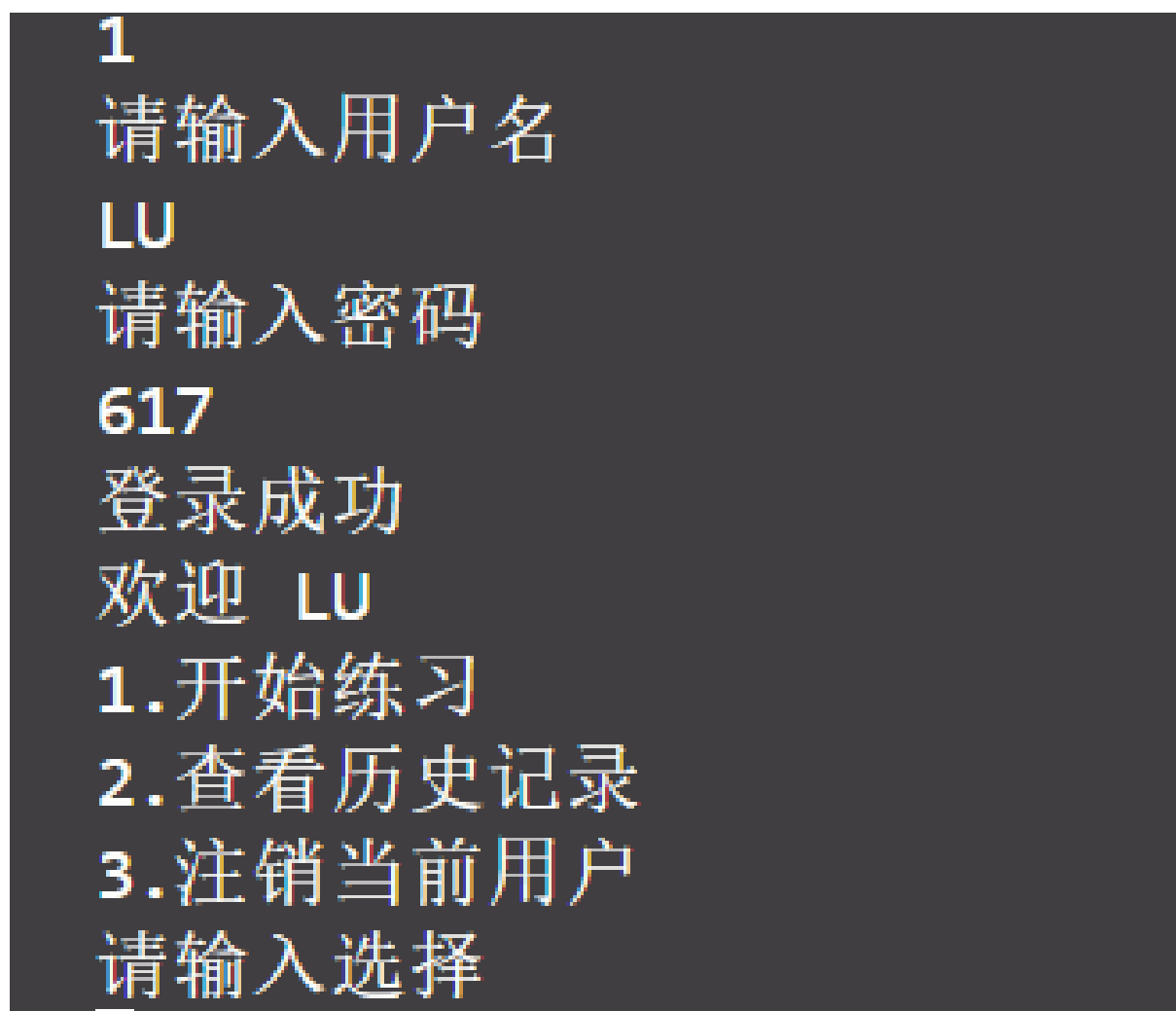
- 功能一：主菜单控制用户注册、登录、退出



- 功能二：注册用户



- 功能三：登录



- 功能四：开始练习

请输入选择
1
请选择难度
1.简单
2.中等
3.困难
1
It is big.
按下回车开始练习

请输入以上文本，注意标点与大小写
It as big.正确字符数:9
正确率: 90.00%
用时: 8.64s
速度: 62.53字/分钟

- 功能五：查看历史记录

历史记录
|时间: 2026-04-10 21:59:41|
|难度: 简单|正确率: 100.00%|速度: 150.55字/分钟|
按回车键继续

3 程序实现中遇到的困难

- 在读取用户输入的练习时，我发现只要在按下回车键开始练习前，误触了其他按键，如按下了”H”，若想要再按回车开始计时程序就直接运行截至。可能还没来得及输入真正的练习文本，程序就已经计算成绩了，不利于用户体验。cin.get() 只读取一个字符，不管它是字母、数字还是回车。按 a 时，cin.get() 读走了'a'，但回车符'\n' 仍然留在输入缓冲区中。紧接着的 getline 看到缓冲区开头就是'\n'，于是直接返回空字符串（getline 的默认行为是遇到换行符就停止，且不保存换行符）。所以需要一行命令清空输入缓冲区里所有残留的垃圾。

```
string sentence = getText(dif);  
cout << sentence << endl;  
cout << "按下回车开始练习" << endl;  
  
while (cin.get() != '\n')  
    ; // 修改后
```

```
cin.get(); // 修改前
```


- 这里使用了 ai 修改，在 HistoryManager 的 loadHistories 中，为什么用 long long t_ll 来读取时间戳，然后转换成 time_t？原因：time_t 类型在不同平台大小可能不同（可能是 32 位整数，也可能是 64 位整数），直接使用 fin » histories[historyCount].time 可能导致跨平台兼容性问题或读取错误。而 long long 是至少 64 位的整数类型，可以安全存储任何时间戳，且文本格式一致。之后再强制转换回 time_t。这是一种安全的做法。

```
string username;
long long t_ll;
int dif;
double ac, speed;
if (ss >> username >> t_ll >> dif >> ac >> speed)
{
    histories[historyCount].username = username;
    histories[historyCount].time = (time_t)t_ll;
    histories[historyCount].dif = dif;
    histories[historyCount].ac = ac;
    histories[historyCount].speed = speed;
    historyCount++;
    if (historyCount >= MAX_History)
        break;
}
}
fin.close();
```

4 程序代码

4.1 Common.h

```
1 #ifndef COMMON_H
2 #define COMMON_H
3
4 #include <string>
5 #include <ctime>
6
7 using namespace std;
8
9 const int MAX_User = 100;
10 const int MAX_History = 500;
11
12 struct user
13 {
14     string username;
15     string password;
16 };
17
18 struct History
19 {
20     string username;
21     time_t time;
22     int dif;
23     double ac;
24     double speed;
25 };
26
27 #endif
```

4.2 Main.cpp

```
1 #include "Game.h"
2 using namespace std;
3
4 int main()
5 {
```

```
6     UserManager userMgr;
7     HistoryManager historyMgr;
8     Game game(userMgr, historyMgr);
9     game.run();
10
11     return 0;
12 }
```

4.3 UserManager.h

```
1 #ifndef USERMANAGER_H
2 #define USERMANAGER_H
3
4 #include "common.h"
5
6 class UserManager
7 {
8 private:
9     user users[MAX_User];
10     int userCount;
11
12 public:
13     UserManager();
14     void loadUsers();
15     void saveUsers();
16     bool registerUser();
17     bool login(string &loggedUsername);
18 };
19
20 #endif
```

4.4 UserManager.cpp

```
1 #include "UserManager.h"
2 #include <iostream>
3 #include <fstream>
4
5 using namespace std;
6
```

```
7  UserManager::UserManager() : userCount(0) {}
8
9  void UserManager::loadUsers()
10 {
11     ifstream fin("users.txt");
12     if (!fin.is_open())
13         return;
14     userCount = 0;
15     while (fin >> users[userCount].username >> users[userCount].
        password)
16     {
17         userCount++;
18         if (userCount >= MAX_User)
19             break;
20     }
21     fin.close();
22 }
23
24 void UserManager::saveUsers()
25 {
26     ofstream fout("users.txt");
27     if (!fout.is_open())
28         return;
29     for (int i = 0; i < userCount; ++i)
30     {
31         fout << users[i].username << " " << users[i].password << endl
            ;
32     }
33     fout.close();
34 }
35
36 bool UserManager::registerUser()
37 {
38     string username, password;
39     cout << "请输入用户名" << endl;
40     cin >> username;
41     for (int i = 0; i < userCount; ++i)
42     {
43         if (users[i].username == username)
44         {
```

```

45         cout << "用户名已存在" << endl;
46         return false;
47     }
48 }
49 cout << "请输入密码" << endl;
50 cin >> password;
51 users[userCount].username = username;
52 users[userCount].password = password;
53 userCount++;
54 cout << "已成功注册" << endl;
55 return true;
56 }
57
58 bool UserManager::login(string &loggedUsername)
59 {
60     string username, password;
61     cout << "请输入用户名" << endl;
62     cin >> username;
63     cout << "请输入密码" << endl;
64     cin >> password;
65     for (int i = 0; i < userCount; ++i)
66     {
67         if (users[i].username == username && users[i].password ==
68             password)
69         {
70             cout << "登录成功" << endl;
71             cout << "欢迎" << username << endl;
72             loggedUsername = username;
73             return true;
74         }
75     }
76     cout << "用户名或密码错误" << endl;
77     return false;
78 }

```

4.5 HistoryManager.h

```

1 #ifndef HISTORYMANAGER_H
2 #define HISTORYMANAGER_H

```

```
3
4 #include "common.h"
5 #include <string>
6
7 class HistoryManager
8 {
9 private:
10     History histories[MAX_History];
11     int historyCount;
12
13 public:
14     HistoryManager();
15     void loadHistories();
16     void saveHistories();
17     void addRecord(const string &username, time_t t, int dif, double
        ac, double speed);
18     void showHistory(const string &username);
19 };
20
21 #endif
```

4.6 HistoryManager.cpp

```
1 #include "HistoryManager.h"
2 #include <iostream>
3 #include <fstream>
4 #include <sstream>
5 #include <iomanip>
6 #include <ctime>
7
8 using namespace std;
9
10 HistoryManager::HistoryManager() : historyCount(0) {}
11
12 void HistoryManager::loadHistories()
13 {
14     ifstream fin("history.txt");
15     if (!fin.is_open())
16         return;
```

```
17     historyCount = 0;
18     string line;
19     while (getline(fin, line))
20     {
21         if (line.empty())
22             continue;
23         stringstream ss(line);
24         string username;
25         long long t_ll;
26         int dif;
27         double ac, speed;
28         if (ss >> username >> t_ll >> dif >> ac >> speed)
29         {
30             histories[historyCount].username = username;
31             histories[historyCount].time = (time_t)t_ll;
32             histories[historyCount].dif = dif;
33             histories[historyCount].ac = ac;
34             histories[historyCount].speed = speed;
35             historyCount++;
36             if (historyCount >= MAX_History)
37                 break;
38         }
39     }
40     fin.close();
41 }
42
43 void HistoryManager::saveHistories()
44 {
45     ofstream fout("history.txt");
46     if (!fout.is_open())
47         return;
48     for (int i = 0; i < historyCount; ++i)
49     {
50         fout << histories[i].username << " "
51             << (long long)histories[i].time << " "
52             << histories[i].dif << " "
53             << fixed << setprecision(2) << histories[i].ac << " "
54             << histories[i].speed << endl;
55     }
56     fout.close();
```

```
57 }
58
59 void HistoryManager::addRecord(const string &username, time_t t, int
    dif, double ac, double speed)
60 {
61     if (historyCount >= MAX_History)
62     {
63         cout << "历史记录已满，无法保存新记录" << endl;
64         return;
65     }
66     histories[historyCount].username = username;
67     histories[historyCount].time = t;
68     histories[historyCount].dif = dif;
69     histories[historyCount].ac = ac;
70     histories[historyCount].speed = speed;
71     historyCount++;
72 }
73
74 void HistoryManager::showHistory(const string &username)
75 {
76     cout << "历史记录" << endl;
77     bool found = false;
78     for (int i = 0; i < historyCount; ++i)
79     {
80         if (histories[i].username == username)
81         {
82             found = true;
83             char timestr[100];
84             struct tm *timeinfo = localtime(&histories[i].time);
85             strftime(timestr, sizeof(timestr), "%Y-%m-%d_%H:%M:%S",
                timeinfo);
86             cout << "|时间：" << timestr << "|" << endl;
87             cout << "|难度：";
88             if (histories[i].dif == 1)
89                 cout << "简单";
90             else if (histories[i].dif == 2)
91                 cout << "中等";
92             else
93                 cout << "困难";
94             cout << "|正确率：" << fixed << setprecision(2) <<
```



```
        histories[i].ac << "%";
95         cout << "|速度: " << fixed << setprecision(2) <<
            histories[i].speed << "字/分钟|" << endl;
96     }
97 }
98 if (!found)
99     cout << "暂无记录" << endl;
100 cout << "按回车键继续" << endl;
101 cin.ignore();
102 cin.get();
103 }
```

4.7 Game.h

```
1  #ifndef GAME_H
2  #define GAME_H
3
4  #include "UserManager.h"
5  #include "HistoryManager.h"
6  #include <vector>
7  #include <string>
8
9  class Game
10 {
11 private:
12     UserManager &userMgr;
13     HistoryManager &historyMgr;
14     std::vector<std::string> easyQuestions;
15     std::vector<std::string> mediumQuestions;
16     std::vector<std::string> hardQuestions;
17
18     void loadQuestions(const std::string &filename, std::vector<std:::
        string> &container);
19     std::string getText(int dif);
20     void cleanInput();
21     void start(const std::string &username);
22     void showMenu(const std::string &loggedUser);
23
24 public:
```

```
25     Game(UserManager &um, HistoryManager &hm);
26     void run();
27 };
28
29 #endif
```

4.8 Game.cpp

```
1  #include "Game.h"
2  #include <iostream>
3  #include <fstream>
4  #include <ctime>
5  #include <cstdlib>
6  #include <algorithm>
7  #include <iomanip>
8  #include <conio.h>
9  #include <windows.h> // 用于控制台颜色
10
11 #define GREEN 10
12 #define RED 12
13 #define WHITE 7
14
15 using namespace std;
16
17 Game::Game(UserManager &um, HistoryManager &hm) : userMgr(um),
18     historyMgr(hm) {}
19
20 void Game::loadQuestions(const string &filename, vector<string> &
21     container)
22 {
23     ifstream fin(filename);
24     string line;
25     while (getline(fin, line))
26     {
27         if (!line.empty())
28             container.push_back(line);
29     }
30     fin.close();
31 }
```

```
30
31 string Game::getText(int dif)
32 {
33     if (dif == 1 && !easyQuestions.empty())
34         return easyQuestions[rand() % easyQuestions.size()];
35     else if (dif == 2 && !mediumQuestions.empty())
36         return mediumQuestions[rand() % mediumQuestions.size()];
37     else if (dif == 3 && !hardQuestions.empty())
38         return hardQuestions[rand() % hardQuestions.size()];
39     else
40         return "No question available. Please check your text files."
41         ;
42 }
43
44 void Game::cleanInput()
45 {
46     cin.clear();
47     cin.ignore(10000, '\n');
48 }
49
50 void Game::start(const string &username)
51 {
52     cout << "请选择难度" << endl;
53     cout << "1. 简单" << endl;
54     cout << "2. 中等" << endl;
55     cout << "3. 困难" << endl;
56     int dif;
57     cin >> dif;
58     if (dif < 1 || dif > 3)
59     {
60         cout << "选择无效" << endl;
61         return;
62     }
63     cleanInput();
64
65     string sentence = getText(dif);
66     cout << sentence << endl;
67     cout << "按下回车开始练习" << endl;
68
69     while (cin.get() != '\n')
```

```
69         ; // 等待回车
70
71     clock_t startTime = clock();
72     cout << "请输入以上文本，注意标点与大小写" << endl;
73     string userInput;
74     int pos = 0;
75     int len = sentence.size();
76     int cor = 0;
77     HANDLE hConsole = GetStdHandle(STD_OUTPUT_HANDLE);
78
79     while (pos < len)
80     {
81         char ch = _getch();
82         if (ch == '\r')
83             break; // 按回车提前结束
84         if (ch == sentence[pos])
85         {
86             SetConsoleTextAttribute(hConsole, GREEN); // 绿色
87             cor++;
88         }
89         else
90         {
91             SetConsoleTextAttribute(hConsole, RED); // 红色
92         }
93         cout << ch;
94         SetConsoleTextAttribute(hConsole, WHITE); // 恢复默认白色
95         userInput.push_back(ch);
96         pos++;
97     }
98     clock_t endTime = clock();
99     double ela = (double)(endTime - startTime) / CLOCKS_PER_SEC;
100
101     double ac = cor * 100.0 / len;
102     double speed = cor / (ela / 60.0);
103     cout << "正确字符数:" << cor << endl;
104     cout << "正确率: " << fixed << setprecision(2) << ac << "%" <<
        endl;
105     cout << "用时: " << fixed << setprecision(2) << ela << "s" <<
        endl;
106     cout << "速度: " << speed << "字/分钟" << endl;
```

```
107
108     historyMgr.addRecord(username, time(nullptr), dif, ac, speed);
109 }
110
111 void Game::showMenu(const string &loggedUser)
112 {
113     int choice;
114
115     while (true)
116     {
117         cout << "1.开始练习" << endl;
118         cout << "2.查看历史记录" << endl;
119         cout << "3.注销当前用户" << endl;
120         cout << "请输入选择" << endl;
121         cin >> choice;
122         if (cin.fail())
123         {
124             cleanInput();
125             cout << "请正确输入!" << endl;
126             continue;
127         }
128         if (choice)
129             switch (choice)
130             {
131                 case 1:
132                     start(loggedUser);
133                     break;
134                 case 2:
135                     historyMgr.showHistory(loggedUser);
136                     break;
137                 case 3:
138                     cout << "已注销" << endl;
139                     return;
140                 default:
141                     cout << "请重新进行有效选择" << endl;
142             }
143     }
144 }
145
146 void Game::run()
```

```
147 {
148     // 初始化随机数种子
149     srand((unsigned int)time(NULL));
150
151     // 加载数据
152     userMgr.loadUsers();
153     historyMgr.loadHistories();
154     loadQuestions("easy.txt", easyQuestions);
155     loadQuestions("medium.txt", mediumQuestions);
156     loadQuestions("hard.txt", hardQuestions);
157
158     int choice;
159     string loggedUser;
160     while (true)
161     {
162         cout << "====TypeGame====" << endl;
163         cout << "1. 登录已有账号" << endl;
164         cout << "2. 注册新用户" << endl;
165         cout << "3. 退出" << endl;
166         cout << "注意! 正确退出才可保存数据!" << endl;
167         cin >> choice;
168         if (cin.fail())
169         {
170             cleanInput();
171             cout << "请正确输入!" << endl;
172             continue;
173         }
174         switch (choice)
175         {
176             case 1:
177                 if (userMgr.login(loggedUser))
178                 {
179                     showMenu(loggedUser);
180                 }
181                 break;
182             case 2:
183                 userMgr.registerUser();
184                 break;
185             case 3:
186                 cout << "正在保存数据并退出" << endl;
```

```
187         userMgr.saveUsers();
188         historyMgr.saveHistories();
189         return;
190     default:
191         cout << "请输入正确的选择" << endl;
192     }
193 }
194 }
```

4.9 easy.txt (部分)

```
1 I am happy.
2 You are kind.
```

4.10 medium.txt (部分)

```
1 I enjoy reading books in the quiet library after school.
2 My parents always prepare delicious breakfast for our family every
  morning.
3 The little girl likes to draw colorful pictures in her sketchbook.
```

4.11 hard.txt (部分)

```
1 Effective communication requires not only clear expression but also
  careful listening to diverse perspectives.
2 The ability to adapt to unexpected challenges is a vital skill in
  both academic and professional environments.
3 Sustainable development aims to meet present needs without
  compromising the ability of future generations to meet their own.
```